❶ The class implements IOutboundParameterTransformer and its TransformOutbound method.

❷ The processing logic simply replaces spaces with hyphens.

This particular implementation is very naive. For instance, it doesn't take account of any existing hyphens in the value being transformed or any double spaces, but it is sufficient to serve as an example. The registration process for parameter transformers that act on route parameter values is very similar to the one used to register your custom route constraint earlier. The parameter transformer is added to the `ConstraintMap` property of the `RouteOptions` object, and it is assigned to the target route parameter in the same way as a route constraint. The next listing shows how the `SlugParameterTransformer` is registered as part of `RouteOptions`.

Listing 4.17 Registering a parameter transformer like a constraint

```
builder.Services.Configure<RouteOptions>(options =>
{
    options.LowercaseUrls = true;
    options.ConstraintMap.Add("city", typeof(CityConstraint));
    options.ConstraintMap.Add("slug", typeof(SlugParameterTransformer));
});
```

Having registered the parameter transformer, you apply it to selected route parameters in the same way as a route constraint with the name that was specified at registration:

```
@page "{name:slug}/{rating?}"
```

NOTE Despite the fact that you registered and applied the parameter transformer in a way similar to a route constraint, the parameter transformer does not act in the same way as a constraint. It plays no part in mapping URLs to route templates.

Now when you rerun the application, the link to the New York page looks a lot better (figure 4.10).